LANGCHAIN — EASY COMPLETE NOTES

Placement + Projects + RAG + Agents + LangGraph

> **Purpose:** Understand the complete flow first. Then learn only the concepts useful for LLM apps, RAG, Agentic RAG, interviews, and projects.

Video compared: “Learn LangChain in 2 Hours — Complete LangChain Course in Telugu — Build RAG & AI Applications.” The supplied YouTube page confirms this title; its accessible page did not expose a detailed transcript, so the comparison uses the video's stated scope rather than claiming exact timestamps.

1. THE FULL FLOW

```
BASIC LANGCHAIN

USER
↓
PROMPT TEMPLATE
↓
LLM / CHAT MODEL
↓
OUTPUT PARSER
↓
ANSWER


RAG

USER QUESTION
↓
RETRIEVER
↓
RELEVANT DOCUMENTS
↓
PROMPT = QUESTION + CONTEXT
↓
LLM
↓
PARSER
↓
ANSWER


AGENTIC RAG

USER
↓
AGENT
↓
DECIDE
■■■ RETRIEVER / VECTOR DB
■■■ SQL
■■■ WEB / API
■■■ OTHER TOOLS
↓
RESULTS
↓
EVALUATE
■■■ ENOUGH → ANSWER
■■■ NOT ENOUGH → ACT / RETRIEVE AGAIN
↓
LOOP
```

2. VIDEO vs THIS ROADMAP

Topic	Video / stated scope	This PDF
Basics	2-hour LangChain introduction	Basics explained simply

Topic	Video / stated scope	This PDF
RAG	Build RAG and AI applications	Full RAG pipeline + retrieval concepts
Prompts / Models / Chains	Core LangChain learning	Covered with simple examples
Documents / Splitting / Embeddings	Relevant to RAG	Covered
Vector stores / Retrievers	Relevant to RAG	Covered
Tools / Agents	Useful for AI apps	Covered
Agentic RAG	Beyond basic RAG	Covered as the next layer
LangGraph	Not verifiable from the page alone	Added as an important stateful-agent topic
Advanced retrieval	Not fully verifiable from the page alone	Hybrid, BM25, MMR, reranking, rewriting, multi-query, parent-child, compression, multi-hop, Self-RAG, CRAG, Graph RAG
Production	Not fully verifiable from the page alone	Evaluation, security, retries, tracing, cost and latency

Conclusion: Use the video for practical exposure. Use this PDF as the organized placement-oriented checklist that connects LangChain → RAG → tools → agents → LangGraph → production.

3. LANGCHAIN BASICS

LLM / Chat Model

The model that generates the response.

```
llm.invoke("Explain RAG")
```

invoke()

Runs a component with the supplied input.

```
invoke = execute / run
```

Prompt Template

Reusable prompt with variables.

```
"Explain {topic} in simple words."
```

LCEL / |

Connects components so data flows from left to right.

```
prompt | llm | parser
```

Chain

A connected sequence of operations.

```
Prompt → LLM → Parser
```

Output Parser

Converts model output into the format the app needs.

```
AI message → string / structured output
```

4. DOCUMENT → RAG FLOW

```
PDF / DOC / WEB / CSV
↓
DOCUMENT LOADER
↓
DOCUMENTS
↓
TEXT SPLITTER
↓
```

```
CHUNKS
↓
EMBEDDINGS
↓
VECTOR STORE
↓
RETRIEVER
↓
RELEVANT CHUNKS
↓
PROMPT + CONTEXT
↓
LLM
↓
ANSWER
```

Document Loader

Loads files or other data into LangChain Document objects.

Document

Usually contains page_content plus metadata.

Text Splitter

Breaks large content into chunks. Overlap helps preserve context across boundaries.

Embeddings

Converts text into vectors so semantic similarity can be measured.

Vector Store

Stores/searches vectors. Examples: Chroma, FAISS, Pinecone, Qdrant.

Retriever

Fetches relevant documents for a query.

5. RETRIEVAL — MUST KNOW

Similarity Search — Find semantically similar chunks using embeddings.

Metadata Filtering — Restrict search using fields such as project, year, type, or tenant.

MMR — Balances relevance and diversity to reduce duplicate results.

BM25 / Keyword Search — Strong for exact words, IDs, names, error codes, and identifiers.

Hybrid Search — Combines keyword search and vector/semantic search.

Reranking — Reorders retrieved candidates using a stronger relevance model.

Query Rewriting — Turns a vague query into a retrieval-friendly query.

Multi-Query Retrieval — Generates several alternative queries and combines their results.

Parent-Child Retrieval — Retrieve a precise child chunk, then return the larger parent context.

Context Compression — Removes irrelevant material before the LLM sees it.

Multi-Hop RAG — Retrieves in dependent steps where one result helps form the next query.

Self-RAG — Evaluates whether retrieval is needed and whether evidence supports the answer.

CRAG — Evaluates retrieval quality and performs corrective retrieval when needed.

Graph RAG — Uses entities and relationships for relationship-heavy questions.

6. CHAIN vs AGENT

```
CHAIN
A → B → C → D
Mostly predefined.

AGENT
Question
↓
Agent decides
↓
Tool A
↓
Observe
↓
Agent decides again
↓
Tool B / Answer
```

Easy rule: Chain = fixed workflow. Agent = dynamic decision-making.

7. TOOLS + TOOL CALLING

```
USER
↓
AGENT / LLM
↓
"I need SQL"
↓
SQL TOOL
↓
DATABASE RESULT
↓
AGENT / LLM
↓
ANSWER
```

Tool: an external capability such as calculator, SQL, search, API, Python, or file operation.

Tool calling: the model requests execution of a tool with arguments; the application executes it and returns the result.

8. REACT

```
QUESTION
↓
DECIDE NEXT ACTION
↓
ACT → TOOL
↓
OBSERVE RESULT
↓
DECIDE AGAIN
↓
ACT AGAIN OR ANSWER
```

Example: “What is 125 × 48?” → choose calculator → receive 6000 → answer.

9. AGENTIC RAG

```
QUESTION
↓
AGENT
↓
DECIDE
■■■ VECTOR RETRIEVER
■■■ SQL
■■■ WEB/API
■■■ OTHER TOOL
↓
RESULTS
↓
EVALUATE
■■■ ENOUGH → ANSWER
```

```
■■■ NOT ENOUGH → REWRITE / RETRIEVE / TOOL
↓
LOOP
```

Example: “Why was Project Alpha delayed and how much did it exceed budget?” The agent can retrieve project reports for causes and use SQL for financial numbers, then combine the evidence.

10. STRUCTURED OUTPUT

Applications often need predictable fields rather than free-form text.

```
{
"answer": "Project Alpha was delayed...",
"sources": ["report_4", "report_7"],
"confidence": 0.91
}
```

11. STATE + MEMORY

State: data carried through the current workflow, such as question, documents, tool results, and answer.

Short-term conversation memory: information needed across turns in the current thread/workflow.

Long-term memory: persistent information stored for future interactions when the application supports it.

```
STATE
■■■ question
■■■ documents
■■■ tool_results
■■■ answer
```

12. LANGGRAPH

LangGraph is useful when an agent needs explicit state, branching, loops, checkpoints, persistence, or human approval.

```
START
↓
ROUTER
↓
RETRIEVE
↓
EVALUATE
■■■ GOOD → GENERATE → END
■■■ BAD → REWRITE → RETRIEVE
↑
■■■ LOOP
```

State — Information carried through the graph.

Node — An operation such as retrieve, call LLM, evaluate, or call a tool.

Edge — A connection from one node to another.

Conditional Edge — Chooses the next node based on a condition.

Checkpoint — Saves workflow state so it can be resumed.

Human-in-the-loop — Pauses for human approval before sensitive actions.

13. PRODUCTION

Retries: recover from temporary LLM/API/tool failures.

Fallbacks: use another model or retrieval route when appropriate.

Streaming: return partial output as it is generated.

Tracing: inspect prompts, retrieval, tool calls, latency, errors, and outputs.

Evaluation: retrieval quality, context relevance, faithfulness, answer relevance, tool/agent success.

Security: prompt injection, least privilege, tool permissions, data leakage, unsafe SQL, tenant isolation.

Cost and latency: control tokens, model calls, retrieval count, and agent loops.

14. INTERVIEW COMPARISONS

Concept	Simple meaning
Prompt Template	Reusable prompt with variables
Chain	Predefined connected workflow
Retriever	Finds relevant documents
Vector Store	Stores/searches vectors
Tool	External capability
Tool Calling	Model requests execution of a tool
Agent	Dynamically decides actions
RAG	Retrieve context, then generate
Agentic RAG	Agent decides how/when to retrieve or use tools
Query Rewrite	One improved query
Multi-Query	Multiple alternative queries
Self-RAG	Self-evaluates retrieval/grounding
CRAG	Corrects weak retrieval
Parent-Child	Precise retrieval + broader context
Compression	Removes irrelevant context

15. FINAL REVISION FLOW

```
LANGCHAIN
↓
PROMPTS + MODELS + CHAINS
↓
DOCUMENTS + RETRIEVERS
↓
RAG
↓
TOOLS + TOOL CALLING
↓
AGENTS
↓
STATE + LANGGRAPH
↓
AGENTIC RAG
↓
PRODUCTION
```

Placement priority: deeply understand the flow and the differences between these components. Do not waste time memorizing every LangChain integration or obscure API.